

AMBI Algorithms

AMBI Algorithms

A collection of algorithms implemented for learning and performance comparison purposes.

Multiplication Algorithms

1. A la Russe (Russian Peasant Multiplication)

A la Russe is an algorithm used for the multiplication of two large integers. It avoids large multiplication tables by using only doubling, halving (floor division by 2), and addition.

How it Works:

1. Take two integers, m and n .
2. In each step:
 - Multiply m by 2.
 - Divide n by 2 using floor division.
3. Continue until n reaches 1.
4. The final result is the sum of all values of m where the corresponding n was odd.

Complexity The algorithm performs $O(\log n)$ doublings and halvings. For n -digit numbers, addition takes $O(n)$, leading to an overall complexity of:

$$T(n) = O(n^2)$$

2. Karatsuba (Divide and Conquer)

Given two large integers of length n (assuming $m \approx n$), the goal is to divide the large multiplication into smaller multiplications.

Classical Divide and Conquer If we divide each n -digit number into two $n/2$ -digit numbers:

$$xy = (10^{n/2}a + b)(10^{n/2}c + d) = 10^n ac + 10^{n/2}(ad + bc) + bd$$

This requires 4 multiplications of $n/2$ -digit numbers (ac, ad, bc, bd). According to the Master Theorem:

$$T(n) = 4T(n/2) + O(n)$$

* $a = 4, b = 2, k = 1$ * Since $a > b^k$ ($4 > 2^1$):

$$T(n) = \Theta(n^{\log_b a}) = \Theta(n^{\log_2 4}) = \Theta(n^2)$$

This is not an improvement over classical long multiplication or the Russian method.

The Karatsuba Optimization Karatsuba discovered that we can compute $(ad + bc)$ using only **one** additional multiplication:

$$(a + b)(c + d) = ac + ad + bc + bd$$

Therefore:

$$ad + bc = (a + b)(c + d) - ac - bd$$

Now we only need **3** multiplications: ac , bd , and $(a + b)(c + d)$.

Example: 1234×456

1. **Zero Padding:** 1234×0456

2. **Split:**

- $w = 12, x = 34$
- $y = 04, z = 56$

3. **Recursive Steps:**

- $p = w \times y = 12 \times 04$
- $q = x \times z = 34 \times 56$
- $r = (w + x) \times (y + z) = (12 + 34) \times (04 + 56) = 46 \times 60$

4. **Combine:** $10^4 p + 10^2(r - p - q) + q$

Complexity

$$T(n) = 3T(n/2) + O(n)$$

* $a = 3, b = 2, k = 1$ * Since $a > b^k$ ($3 > 2^1$):

$$T(n) = \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$$

Sorting Algorithms

1. Insertion Sort

Insertion Sort involves comparing each element in the array to its predecessors and adjusting it by “inserting” it into its correct position.

```
function insert(T[1...n])
  for i <- 2 to n do
    x <- T[i]
    j <- i - 1
    while j > 0 and x < T[j] do
      T[j+1] <- T[j]
      j <- j - 1
    T[j+1] <- x
  return T
```

Pseudocode:

Complexity:

- **Worst Case:** $O(n^2)$
 - **Best Case:** $\Omega(n)$
-

2. Selection Sort

Selection Sort involves finding the minimum element from the unsorted part of the array and putting it at the beginning.

```
function select(T[1...n])
  for i <- 1 to n-1 do
    min_idx <- i
    min_val <- T[i]
    for j <- i+1 to n do
      if T[j] < min_val then
        min_idx <- j
        min_val <- T[j]
    T[min_idx] <- T[i]
    T[i] <- min_val
  return T
```

Pseudocode:

Complexity:

$$T(n) = O(n^2)$$

3. Heap Sort

Heap Sort works by first converting the array into a binary heap.

```
function heapsort(T[1...n])
  make_heap(T)
  for i <- n down to 2 do
    swap(T[1], T[i])
    sift_down(T, 1, i-1)
  return T
```

Pseudocode:

Complexity:

$$T(n) = O(n \log n)$$

4. Merge Sort (Divide and Conquer)

Merge Sort is a recursive algorithm that divides an array into halves, sorts them independently, and then merges the results.

How it Works:

1. **Divide:** Split the array into two (roughly) equal halves.
2. **Conquer:** Recursively sort each half.
3. **Combine:** Merge the two sorted halves into a single sorted array.

```
function mergesort(T[1...n])
  if n > 1 then
    q <- floor(n / 2)
    U <- T[1...q]
    V <- T[q+1...n]
    mergesort(U)
    mergesort(V)
    merge(U, V, T)
  return T

function merge(U, V, T)
  i <- 1; j <- 1
  U[length+1] <- Infinity
  V[length+1] <- Infinity
  for k <- 1 to length(T) do
    if U[i] < V[j] then
      T[k] <- U[i]; i <- i + 1
    else
      T[k] <- V[j]; j <- j + 1
  return T
```

Pseudocode:

Complexity:

$$T(n) = \Theta(n \log n)$$

5. Quicksort (Divide and Conquer)

Quicksort is a highly efficient sorting algorithm that uses a “pivot” element to partition the array into two sub-arrays.

How it Works:

1. **Pivot Selection:** Choose an element from the array (e.g., the first element).
2. **Partitioning:** Rearrange the array so that all elements smaller than or equal to the pivot are on the left, and all elements larger than the pivot are on the right.
3. **Recursion:** Recursively apply the same logic to the left and right sub-arrays.

```
function quicksort(T[i...j])
  if j < 5 then // array T is sufficiently small, use insertion sort
    insert(T)
  else
    l <- partition(T, i, j)
    quicksort(T, i, l - 1)
    quicksort(T, l + 1, j)
  return T

function partition(T[i...j])
  p <- T[i] // Using first element as pivot
  k <- i; l <- j + 1
  repeat
    repeat k <- k + 1 until T[k] > p or k >= j
    repeat l <- l - 1 until T[l] <= p
    if k < l then
      swap(T[k], T[l])
  until k >= l
  swap(T[i], T[l]) // Place pivot in correct position
  return l // Return partition index
```

Pseudocode:

Complexity:

- **Average Case:** $T(n) = \Theta(n \log n)$
- **Worst Case:** $T(n) = O(n^2)$ (occurs when the pivot consistently results in extremely unbalanced partitions).

```
python quick_sort.py
```

```
QuickSort called on indices [0:6] → [5, 2, 8, 1, 7, 10, 3]
Initial pivot is: 5
Pointer k initiated at 0
Element here is 5
Pointer l initiated at 7
Currently no element here (beyond array)
pointer k moved to 1
Element here is 2
pointer k moved to 2
```

```

Element here is 8
pointer l moved to 6
Element here is 3
3 and 8 swapped
Array is now [5, 2, 3, 1, 7, 10, 8]
pointer k moved to 3
Element here is 1
pointer k moved to 4
Element here is 7
pointer l moved to 5
Element here is 10
T[l] is 10 and p is 5
pointer l moved to 4
T[l] is 7 and p is 5
pointer l moved to 3
k is now 4 and l is now 3. Crossover
1 and 5 swapped
Array is now [1, 2, 3, 5, 7, 10, 8]
Pivot placed at index 3, value 5
Left partition: [1, 2, 3]
Right partition: [7, 10, 8]
Array is sufficiently small. Using insertion sort now
Array is sufficiently small. Using insertion sort now
Sorted array: [1, 2, 3, 5, 7, 8, 10]

```

Example execution:

Change Making Problem

Problem definition: Pay back change in the fewest number of coins possible.

1. Greedy Change Making

Assumption: an unlimited number of coins is available. At each step, the coin with the highest possible value is used (greedy selection). This may not always lead to an optimal solution; the optimum may require skipping a greedy step.

```

function makechange(n)
  C = {100, 25, 10, 5, 1} // Set of candidates (available coins for selection)
  S <- {} // initialized empty solution set
  s <- sum of items in S
  while s != n
    x <- largest item in C such that x + s <= n // selection function (largest item); feasibility f
    if no such item

```

```

        return no solution found
    S <- S \bigcup {a coin of value x}
    s <- s + x
return S // Solution function

```

Pseudocode:

Complexity

- **Time Complexity:** $O(k \cdot m)$, where k is the number of coin denominations (e.g., 5) and m is the number of coins in the returned solution.
 - In the worst case where only 1-value coins are used, $m = n$, meaning the loop runs n times giving $O(n)$.
 - (Note: This greedy algorithm can be optimized to $O(k)$ time if integer division is used instead of adding coins one by one).
 - **Space Complexity:** $O(m)$ to store the solution set S , where m is the number of coins given as change.
-

2. Change Making with Dynamic Programming

The algorithm computes optimal solutions for all smaller subinstances and combines them to obtain the optimal solution for the full problem.

How it works:

1. A matrix C is constructed where:
 - rows represent coin denominations
 - columns represent amounts from 0 to N
2. Each entry $C[i, j]$ stores the **minimum number of coins** needed to make amount j using the first i coin types.
3. The update rule is:

$$C[i, j] = \min \begin{cases} C[i-1, j] \\ 1 + C[i, j - d_i] \end{cases}$$

- $C[i-1, j]$: do **not** use coin d_i
 - $1 + C[i, j - d_i]$: use coin d_i
4. The optimal solution is found at $C[n, N]$.
 5. Backtracking through the matrix determines which coins were used.

```

function change(N)
    array d[1...n]           // coin denominations
    array C[1...n, 0...N]    // DP matrix

    for i <- 1 to n do
        C[i,0] <- 0          // amount 0 needs 0 coins

    for i <- 1 to n do
        for j <- 1 to N do

            if i = 1 and j < d[1] then
                C[i,j] <- + \infty

            else if i = 1 then
                C[i,j] <- 1 + C[1, j - d[1]]

            else if j < d[i] then
                C[i,j] <- C[i-1,j]

            else
                C[i,j] <- min( C[i-1,j],
                               1 + C[i, j - d[i]] )

    return C

function backtrack(N)
    array d[1...n] // Coin denominations
    C = change(N)
    i <- n
    j <- N
    while j > 0 do
        if i > 1 and C[i,j] = C[i-1,j] then
            i <- i - 1 // No coin of denom i needed
        else
            print d[i] // coin i used
            j <- j - d[i]

```

Pseudocode:

Complexity

- DP matrix contributes to a complexity of $n \times (N + 1)$
- Backtracking worst-case would be $O(N)$ with one step per coin used.
- Overall complexity is $O(n \cdot N)$

Knapsack Problem

Given a knapsack of capacity W and a set of items, each with value v_i and weight w_i , the goal is to maximize the total value of items placed in the knapsack without exceeding the capacity.

1. Greedy Approach with Fractions (Fractional Knapsack)

In this variant, items can be taken in **fractions**, allowing greater flexibility in achieving the optimal value.

Each item contributes:

$$x_i \cdot v_i$$

where:

- x_i is the fraction of item i taken
- $0 \leq x_i \leq 1$

Objective Function

$$\max \sum_{i=1}^n x_i v_i$$

Constraints

$$\sum_{i=1}^n x_i w_i \leq W$$

$$0 \leq x_i \leq 1 \quad \forall i$$

$$v_i > 0, \quad w_i > 0$$

```
function Knapsack(w[1..n], v[1..n], W)
  for i <- 1 to n do
    x[i] <- 0 // fraction for each object initialized to 0
  total_weight <- 0 // knapsack is initially empty

  while total_weight < W and items remain do
    i <- best remaining object // greedy selection (highest v[i]/w[i])
    if total_weight + w[i] <= W then // adding whole object i is feasible
      x[i] <- 1 // take the whole item
      total_weight <- total_weight + w[i]
    else
      x[i] <- (W - total_weight) / w[i] // take a fraction
      total_weight <- W // knapsack is now full

  remove object i from consideration

  return x // return set of fractions
```

Pseudocode

Complexity

- Ratio computation - $O(n)$
 - Sorting of the ratios to pick the best ratio ($O(n \log n)$)
 - Greedy loop gives $O(n)$ because at worst it goes over n items
 - *(Only one item (the final item added to the knapsack) is taken as a fraction to fill the remaining space. Everything else before was taken as a whole)*
-

2. Knapsack with Dynamic Programming

The **0/1 Knapsack Problem** restricts items to being either fully taken or completely left behind (no fractions allowed). Dynamic Programming (DP) is used to guarantee an optimal solution.

How it works:

1. A matrix V is constructed where:
 - rows i represent the set of items from 1 to n .
 - columns j represent the available knapsack capacities from 0 to W .
2. Each entry $V[i, j]$ stores the **maximum value** achievable using a subset of the first i items, with a weight limit of w .
3. The update rule is:
 - If the current item's weight exceeds the capacity ($w_i > w$):
$$V[i, j] = V[i - 1, j]$$
(the item cannot be included)
 - Otherwise, we take the maximum of excluding or including the item:
$$V[i, j] = \max\{V[i - 1, j], v_i + V[i - 1, j - w_i]\}$$
4. The maximum total value is found at $V[n, W]$.

```
function knapsack(W)
    array w[1..n] //array of weights for each item
    array v[1..n] //array of values for each item
    array V[0..n, 0..W] //DP matrix
    for i <- 0 to n do
        for j <- 0 to W do
            if i = 0 or j = 0 then
                V[i,j] <- 0
            else if w[i] <= j then // the item fits
                V[i,j] <- max(V[i-1,j] //Don't take current item
                             v_i + V[i-1,j-w[i]] //Take current item)
            else
                V[i,j] <- V[i-1,j]
    return V[n,W]
```

Pseudocode:

Complexity

- **Time Complexity:** $O(n \cdot W)$, where n is the number of items and W is the maximum knapsack capacity. The two nested loops iterate exactly this many times.
- **Space Complexity:** $O(n \cdot W)$ to store the 2D DP matrix. (*Note: This can be optimized to $O(W)$ by storing only the previous and current rows, though full backtracking to find the exact items would then require the full matrix*).

Spanning Trees (minimum/maximum)

Given a weighted graph, the purpose of these algorithms is to find the optimal combination of edges that gives the minimum or maximum possible sum of weights that connects all nodes with no cycles.

1. Kruskal's Algorithm - minimum spanning tree (greedy)

- Takes set of edges as input and at each step picks the optimal edge (with the lowest weight) and adds to the graph
- *(Compares connected components(remaining edge candidates should not be in a connected component-prevents cycles))*
- It creates a spanning forest(while still not at the solution), and compares non considered edges to join connected components Eventually ends with a minimum spanning tree
- *(If the graph is not fully connected it results in a spanning forest)*

```
function kruskal(weighted_graph_G)
    sort(E(G)) //sort the edges in order of increasing cost/weight -heap sort hear results in o (nlog n)
    n <- |V(G)| initialize n sets // each node starts as its own connected component
    T <- {} empty set of edges //compiles the selected edges
    repeat
        e <- {u,v} //select edge with the smallest weight
        ucomp <- find(u) //given the selected edge, what is u(start node)
        vcomp <- find(v) // '' '' '' '' , what is v(end node)
        if ucomp != vcomp then // the two end nodes occur in different connected components
            merge(ucomp,vcomp) // join the two compenents to one connected component using the select
            T <- T \bigcup {e} //add the edge to the solutin set
    until T contains n-1 edges // A graph contains n-1 edges. stopping condition means iteration over
```

Pseudocode

Complexity

- Sorting the edges takes $O(m \log m)$ where $m = |E|$. With $m = O(n^2)$ worst-case and using a binary heap this is $O(m \log n)$.
- Each edge is processed once in the main loop. Find/union operations using disjoint-set with path compression and union by rank take amortized $O(\alpha(n))$, where α is the inverse Ackermann function.
- Total time complexity: $O(m \log n + m\alpha(n))$, commonly simplified to $O(m \log n)$ for sparse graphs and near-linear in practice.
- Space complexity: $O(n + m)$ to store the graph and disjoint-set structures.

Notes: if the graph is disconnected, Kruskal returns a minimum spanning forest.

2. Prim's Algorithm - minimum spanning tree(greedy)

Builds a spanning tree from an arbitrarily selected node in the graph (any node will result in the same spanning tree) Works with both the edge and node set (kruskals iterates through the edge set but still has to find nodes that form the edge components)

How it works

1. Given a starting node (kept in a set if already considered nodes), all incident edges to the node are compared and the edge with the minimum weight (plus the node on the other end) is added to the tree.
2. iteratively checks all other nodes adjacent to all the nodes of the established spanning tree, adds the one with the lowest weight, until the set of already considered nodes is the same as the set of all the nodes of the graph

```
function prim(weighted_graph_G)
  T <- {} // empty set of edges
  B <- {1} // set with an arbitrary starting node - any node in the graph
  while B != V(G) do // all nodes become included, O(n-1) iterations
    find e = {u,v} of min cost s.t. $u \in B$ and $v \in V(G) \setminus B$ // add smallest frontier
    T <- T \bigcup {e}
    B <- B \bigcup {v}
  return T
```

Pseudocode

Complexity

- Using an adjacency matrix, each step scans $O(n)$ candidate edges and there are n steps, so $O(n^2)$.
- Using a min-priority queue (binary heap) on adjacency lists gives $O((n + m) \log n)$, where $m = |E|$.
- Special cases:
 - Dense graph $m = \Theta(n^2)$ leads to $O(n^2)$ and matrix implementation is acceptable.
 - Sparse graph $m = O(n)$ leads to $O(n \log n)$ with heap-backed adjacency lists.

- Space complexity: $O(n + m)$ for graph and priority queue.

Note: Prim is guaranteed to find an MST if all weights are non-negative, and it grows the tree one vertex at a time.

Shortest Paths

- Given a starting node (or any arbitrary starting node), the goal is to find the shortest path(s) to all other nodes in the graph.
 - A special path is the path from a source to the most distant node (which is the shortest path to that node), with all intermediate nodes as well as the start and end nodes included in the set of graph nodes.
-

1. Dijkstra's Algorithm(Greedy)

Finds shortest paths from a given source node to all other nodes in the graph

How it works:

1. Takes as an argument the adjacency matrix.
2. Initializes an array D of distances from the source node to all other nodes. A min-heap implementation improves the complexity of extracting the minimum.
3. Iteratively checks whether the distance to a node can be reduced via another node, and updates the distance of that node accordingly; selected nodes are removed from the candidate set. This is executed $n - 2$ times.

- *The greedy approach requires non-negative weights.*

```
function Dijkstra(L[1..n, 1..n])
  array D[2..n] //initialize distance array - source to other nodes
  C <- {2..n} //initailize set of candidates (other nodes)
  for i <- 2 to n do
    D[i] <- L[1,i] //populate the distance array with respective distances
  repeat n-1 times // minimization loop (one node chosen per iteration until all reachable nodes are
    v <- some element minimizing D[v] // the smallest distance in D among candidates
    C <- C - {v} // shortest distance to this node is therefore known, remove from candidate set
    for each $w \in C$ do // For the remaining candidates, does going through this node node minim
      D[w] <- \min(D[w], D[v] + L[v,w])
  return D
```

Pseudocode

Complexity

- With an adjacency matrix and simple array/extraction of minimum, time is $O(n^2)$.
- With adjacency list + binary heap (priority queue), time is $O((n + m) \log n)$, where $m = |E|$.
- Space complexity: $O(n + m)$ for graph representation and distance structure.

Note: Dijkstra requires non-negative edge weights; for graphs with negative weights, use Bellman-Ford or Johnson's algorithm.

2. Floyd's Algorithm(Dynamic Programming)

- Finds the shortest path between all pairs of nodes for a given directed weighted graph (no starting node required)
- Takes an adjacency matrix as input and calculates and populates a distance matrix (initially the same as the adjacency matrix) with the shortest distance between node pairs (matrix is updated across multiple iterations -how many?) and an auxiliary P matrix that tracks the order of changes and indicates the nodes whose shortest paths have intermediate nodes
- - **Assumption:** *If k is on the shortest path from i to j , then i to k and k to j are also optimal*
- No loops(diagonals have a 0 entry)
- None edges are represented by a $+\infty$

```
function Floyd(A[1..n, 1..n])
    array D[1..n, 1..n]
    D <- A
    for k <- 1 to n do //all nodes are checked if they can be intermediates with shorter distance between
        for i <- 1 to n do
            for j <- 1 to n do
                D[i,j] = min(D[i,j],
                             D[i,k] + D[k,j])
    return D

function FloydPaths(A[1..n, 1..n])
    array D[1..n, 1..n], P[1..n, 1..n]
    D <- A, P <- 0
    for k <- 1 to n do
        for i <- 1 to n do
            for j <- 1 to n do
                if D[i,j] > D[i,k] + D[k,j]
                    D[i,j] <- D[i,k] + D[k,j]
                    P[i,j] <- k
    return D, P
```

Pseudocode

Complexity

- iterating through the entire set of nodes 3 times results in a complexity of $O(n^3)$
 - Dijkstra has complexity $O((n + m) \log n)$; in sparse graphs $m = O(n)$ this becomes $O(n \log n)$. It computes single-source shortest paths.
 - Running Dijkstra for all sources gives $O(n \cdot (n + m) \log n)$ and is less efficient than Floyd-Warshall on dense graphs.
 - Dijkstra is typically better for sparse graphs, while Floyd-Warshall is better for dense graphs.
-

Biological sequences

- Goal: compare biological sequences presented as strings.
 - Two strings may have a Hamming distance or an edit distance; the latter is the smallest number of individual alterations (insertions and deletions) required to transform a string A into another string B.
 - String comparison can be expressed as:
 - **Alignment:**
 - * 2 rows of strings with columns showing matches/mismatches and insertions/deletions.
 - * number of symbols present up to a given position (gaps appear as repeated or non-increasing counts).
 - **Edit Graph:**
 - * a 2-D matrix representation of the alignment, with edges showing alignment transitions.
 - * it is a directed acyclic graph with the starting node having no incoming edges and the end node having no outgoing edges.
-

1. Longest Common Subsequence

- It is the total number of shared characters (matches) between two sequences; it indicates the points where the sequences coincide.
- In LCS, only matches are scored (1); everything else scores 0.
- Edit distance is given by $m + n - 2 \cdot |\text{LCS}(v, w)|$.
 - *At worst, there are zero matches, so the entire lengths of both sequences are aligned with gaps.*
 - *A single match reduces the number of columns by 2 (if mismatched it would take two separate columns).*

$s_{i,j}$ denotes the length of the LCS between the prefix of v with length i ($v_1 \dots v_i$) and the prefix of w with length j ($w_1 \dots w_j$).

$$s_{i,0} = 0 \quad 0 \leq i \leq n \quad s_{0,j} = 0 \quad 0 \leq j \leq m$$

Update rule:

$$s_{i,j} = \begin{cases} s_{i-1,j-1} + 1 & \text{if } v_i = w_j, \\ \max(s_{i-1,j}, s_{i,j-1}) & \text{otherwise.} \end{cases}$$

Solution is $s_{n,m}$.

```

function LCS(v,w)
  n <- length(v)
  m <- length(w)
  array S[0..n, 0..m]
  array b[0..n, 0..m] // backtracking matrix
  for i <- 0 to n do
    S[i,0] <- 0
  for j <- 0 to m do
    S[0,j] <- 0
  for i <- 1 to n do
    for j <- 1 to m do
      if (v_i = w_j) then
        q <- 1 // only a match increments LCS
      else
        q <- 0
      S[i,j] <- max (S[i-1,j],
                    S[i,j-1],
                    S[i-1,j-1] + q)

      if S[i,j] = S[i-1,j-1] + q then
        b[i,j] <- 3 // prefer diagonal when match/optimal
      else if S[i,j] = S[i-1,j] then
        b[i,j] <- 1 // from up: gap in w
      else if S[i,j] = S[i,j-1] then
        b[i,j] <- 2 // from left: gap in v
  return [n,m] // last cell stores length of LCS

```

Pseudocode Auxiliary matrix required for backtracking - Use the matrix **b** to reconstruct the LCS via `printLCS`; start from the bottom-right corner in the cell `[n,m]` and follow pointers to reconstruct the matched characters.

```

function PrintLCS (b,v,i,j) //only one of the sequences needed as input for printing out the LCS
  if i = 0 or j = 0
    return
  if b[i,j] = 3 then, was a match
    PrintLCS(b,v,i-1,j-1)
    print(v_i)
  else if b[i,j] = 1
    PrintLCS(b,v,i-1,j) can go up to m upward steps until i = 0
  else
    PrintLCS(b,v,i,j-1) can go up to n leftward steps until j = 0

```

The output is not an alignment but a single sequence of shared matches between two other candidate sequences

Complexity

- comparing all characters of two strings of length n and m results in a complexity of $O(n \cdot m)$ or $O(n^2)$ if the strings are of equal length

2. Global Alignment (Needleman-Wunsch)

- Sequences are compared along their entire lengths (global alignment).
- Corresponds to a path from source to sink in an edit graph.
- Similar to LCS but uses match/mismatch scores and gap penalties as edge weights (not just 1 for match and 0 otherwise).
- mismatch scores can be obtained from substitution matrices which have different alignment scores for individual bases based on evolutionary evidence of the most likely substitutions
 - BLOSUM - which uses scores that are calculated based on estimated sequence similarity
 - PAM - which uses scores that are calculated based on estimated sequence divergence
- The update rule for global alignment then gets modified to:

$$S_{i,j} = \max \begin{cases} S_{i-1,j} + \delta(v_i, -') \\ S_{i,j-1} + \delta(' -', w_j) \\ S_{i-1,j-1} + \delta(v_i, w_j) \end{cases}$$

3. Local Alignment (Smith-Waterman)

- Alignments are based on regional similarity/homology
- Accounts for potential deletions of entire regions(substrings)
- Allows any node to be a potential starting node and the optimal score to be anywhere in the graph by having 0 weighted edges from $S[0,0]$ to any other node in the graph(the max will not always be at $S[n,m]$)
 - *May also have multiple optimal alignments- selection would depend on specified criteria*
- Only a minimal change to the update rule:

$$S_{i,j} = \max \begin{cases} 0 \\ S_{i-1,j} + \delta(v_i, -') \\ S_{i,j-1} + \delta(' -', w_j) \\ S_{i-1,j-1} + \delta(v_i, w_j) \end{cases}$$

Gap Handling

- Gaps penalized individually do not usually produce evolutionarily plausible alignments.
- We need to account for region incompatibility in string pairs (some domains may not be shared by both strings) by allowing longer consecutive gaps.
- This is done with affine gap penalties.
- A naive representation would use long edges for longer gaps, increasing the graph size to $O(n^3)$.
- The standard solution is affine gap penalties (**Gotoh Algorithm**).
- This uses a higher cost for opening a gap than extending it. A gap can occur in one of three ways:
 - Coming from a match/mismatch to a gap in either one or both sequences
 - Coming from an already existing gap in one sequence and extension by adding another gap
 - Coming from a gap in one and no gap in the other, followed by a gap opening in the latter
- This is handle by using three matrices:

- M to store alignment scores ending in matches
- V to store alignment scores ending with a gap in V
- W to store alignment scores ending with a gap in W
- The update rule for the matrices then becomes:

$$M_{i,j} = \delta(v_i, w_j) + \max \begin{cases} M_{i-1,j-1} \\ V_{i-1,j-1} \\ W_{i-1,j-1} \end{cases}$$

$$V_{i,j} = \max \begin{cases} M_{i,j-1} + \text{gap start} \rightarrow \text{prev alignment ends in match/mismatch. new gap in v} \\ V_{i,j-1} + \text{gap extend} \rightarrow \text{prev alignment ends in gap in v. gap extension} \\ W_{i,j-1} + \text{gap start} \rightarrow \text{prev alignment ends in gap in w. new gap in v} \end{cases}$$

$$W_{i,j} = \max \begin{cases} M_{i-1,j} + \text{gap start} \rightarrow \text{prev alignment ends in match/mismatch. new gap in w} \\ V_{i-1,j} + \text{gap start} \rightarrow \text{prev alignment ends in gap in v. new gap in w} \\ W_{i-1,j} + \text{gap extend} \rightarrow \text{prev alignment ends in gap in w. gap extension} \end{cases}$$